

DATA-STRUCTURES & ALGORITHMS

with

Manoj Prabhakaran

Lecture 21

Non-Comparison-Based Sorting

Sorting

- So far:
 - Several comparison-based sorting algorithms
 - Merge sort, quicksort, heap sort, tree sort : $O(n \log n)$ time
 - Insertion sort, selection sort, bubble sort : $O(n^2)$ time
 - Sorting networks
 - Odd-even transposition: $O(n)$ depth with $O(n^2)$ comparators
 - $O(\log n)$ depth possible, $O(\log^2 n)$ depth practical
 - Lower Bound:
 - Any comparison-based sorting requires $\Omega(n \log n)$ comparisons
- Today: Non-comparison-based sorting: Counting sort, bucket sort, radix sort

(Basic) Counting Sort

- When the domain of keys to sort is small -- say, $[0, \text{ulim})$

```
void basic_counting_sort (std::vector<unsigned int>& A, unsigned int ulim) {  
    std::vector<int> counts(ulim,0); // one index per key, initialised to 0  
    for(auto x : A)                  // compute count of each key in  $O(n)$  time  
        ++counts[x];  
    for(unsigned int x=0, index=0; x < ulim; x++) // for each key x in domain  
        for(; counts[x] > 0; counts[x]--) // write counts[x] copies of x into A  
            A[index++] = x;               // keeping track of next index to use  
}
```

- Extends to values in a small range $[\text{llim}, \text{ulim})$ (Exercise)
- Let $k = \text{ulim} - \text{llim}$ be the size of the domain of the keys
- $O(n+k)$ time. For small k , beats the $\Omega(n \log n)$ lower bound for comparison sorts!
- $O(k)$ auxiliary space

Collect Instead of Count

- When the items are structured, but with integer keys from a small range
 - Instead of just keeping the count of each key, we need to collect all the items with that key

```
struct item { unsigned int key; /* more data */ };  
void collection_sort (std::vector<item>& A, unsigned int ulim) {  
    std::vector<std::queue<item>> collection(ulim); //per key a queue  
    for(auto& x : A) //for each item in A (without making a copy)  
        collection[x.key].push(std::move(x)); //move it into collection  
    for(unsigned int x=0, index=0; x < ulim; x++) //for each key x  
        for(; !collection[x].empty(); collection[x].pop()) //for each item for x  
            A[index++] = std::move(collection[x].front()); //move it into A  
}
```

- Stable (by collecting in a queue for each key)

Counting Sort : General Version

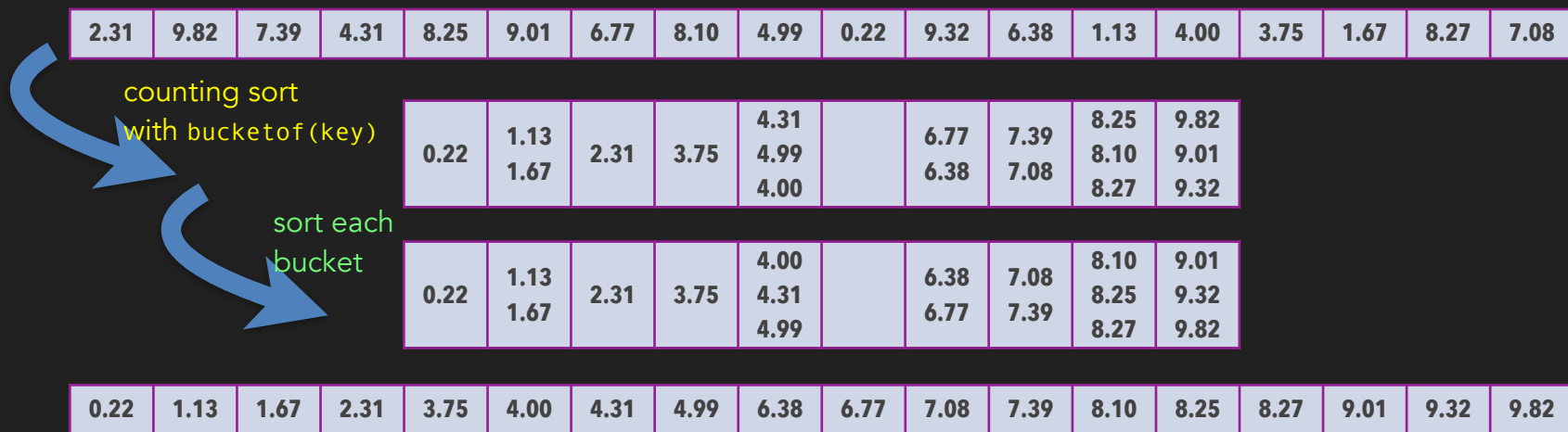
- Use counts to directly find the index in the output (no queues)

```
struct item { unsigned int key; /* more data */ };  
void counting_sort (std::vector<item>& A, unsigned int ulim) {  
    std::vector<unsigned int> offset(ulim,0);    // used for counting  
    std::vector<item> output(A.size());  
    for(auto& x : A) ++offset[x.key]; // after this, offset has counts  
    for(unsigned int i=0, sum = 0; i < ulim; i++) {  
        unsigned int tmp = offset[i];  
        offset[i] = sum; sum += tmp;    // offset[i] has sum of previous counts  
    }  
    for(auto& x : A) // output[ offset[i] ] is where next item with key i goes  
        output [ offset[x.key]++ ] = std::move(x); // move items from A to output  
    A = std::move(output);    // finally move output vector to A  
}
```

- $O(n+k)$ time and auxiliary space. Stable.

Bucket Sort

- Can we generalise counting sort to large key domains (e.g., all of `int`, or `float`)?
 - By *bucketing*! `bucketof(key)` sends key to a small domain of "buckets"
 - Sort items into their respective buckets
 - Using counting sort: with `bucketof(key)` as the key
 - Within each bucket, sort (using any sorting algorithm)



Bucket Sort

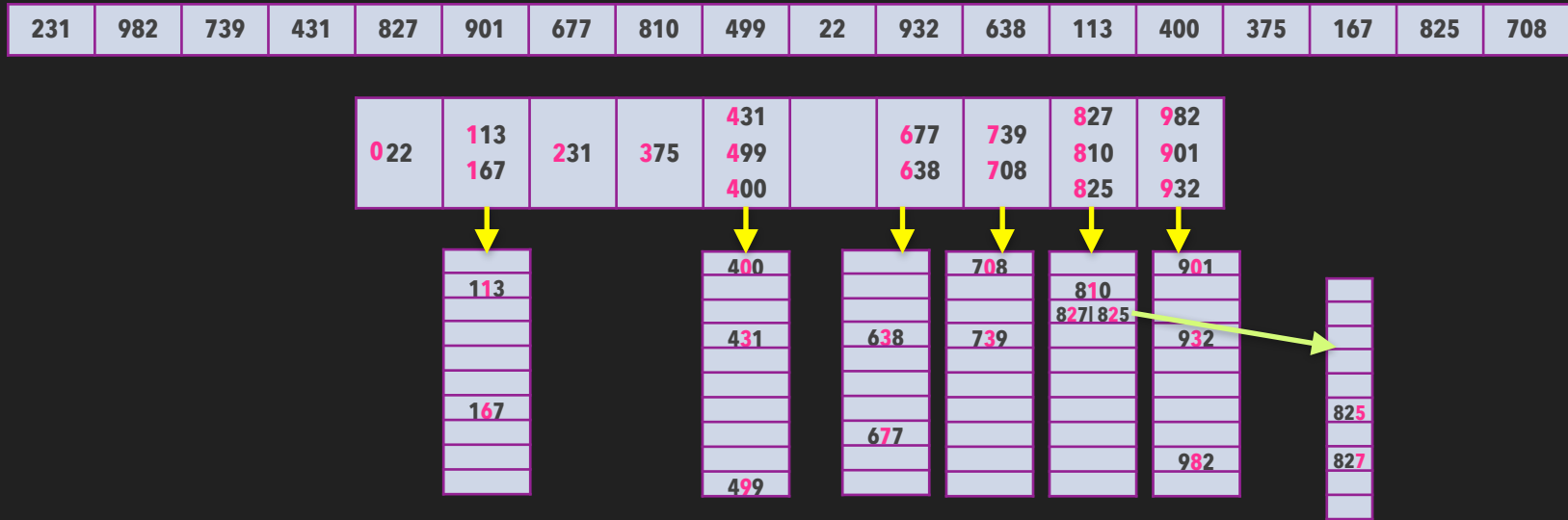
- If we use k buckets, takes $O(n+k)$ auxiliary space
- Total time for sorting the buckets (using a comparison sort)?
 - If $n = O(k)$ and each bucket has $O(1)$ items, total time is $O(n)$
 - E.g., when the keys are uniformly spread throughout its domain
 - But can be as bad as $\Omega(n \log n)$ if one bucket has $\Theta(n)$ items
- Parallelisable
 - Can collect items into buckets in parallel
 - Can sort each bucket in parallel
 - Can copy back from buckets into a vector in parallel (after offsets computed)

Bucket Sort

- What if $\Theta(n)$ items fall into one bucket?
 - If using a comparison based sorting algorithm for sorting within buckets, takes $\Theta(n \log n)$
- Can we still beat comparison based sorting?
 - Can use application-specific heuristics for defining buckets to avoid crowded buckets
 - Or, **recursively, use bucket sort**?
 - When a bucket has "too many" items, split it into smaller buckets
 - If the keys are "granular" (e.g., integers), not too many levels of recursion
 - Leads to another sorting algorithm! **MSD Radix Sort**

Most-Significant-Digit Radix Sort

- Consider d-digit numbers
 - First collect by MSD



- Each bucket with more than one item is sorted using the remaining digits